

Java Collection Framework

Implementation of Important Methods

1. Basic Information

Experiment No	06
Date	02/10/2026
Title	Java Collection Framework - Implementation of Important Methods

2. Objective / Aim

To implement and observe the important methods of the Java Collection Framework classes (ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap and Hashtable) and to understand how ordering, duplicates, sorting and synchronization differ between them.

3. System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

4. Algorithm / Logic Description

- 1 Import the "java.util" package, which contains all collection classes.
- 2 Create a public class named "CollectionDemo" with one static method for each collection class.
- 3 List classes (ArrayList, LinkedList, Vector, Stack): create the object, add elements, then use get(), set(), remove(), contains(), indexOf(), size() and the class-specific methods (addFirst(), poll(), push(), pop(), peek(), search()).
- 4 Set classes (HashSet, LinkedHashSet, TreeSet): add elements including a duplicate, and observe that duplicates are rejected. Observe unordered (HashSet), insertion (LinkedHashSet) and sorted (TreeSet) order. Use first(), last(), higher(), lower(), ceiling(), floor(), pollFirst() and pollLast() on TreeSet.
- 5 Queue classes (PriorityQueue, ArrayDeque): insert elements using add()/offer()/addFirst()/addLast(), inspect with peek(), and remove with poll().
- 6 Map classes (HashMap, LinkedHashMap, TreeMap, Hashtable): store key-value pairs using put(), read with get() and getOrDefault(), and traverse using keySet(), values() and entrySet().
- 7 Use containsKey(), containsValue(), remove(), size(), isEmpty() and clear() to manage the maps.
- 8 Call every demo method from main() and print the result of each operation.
- 9 Compare the outputs to confirm the behaviour of each collection.

5. UML Class Diagram

Figure 1: Collection Framework overview (interfaces and their implementing classes)

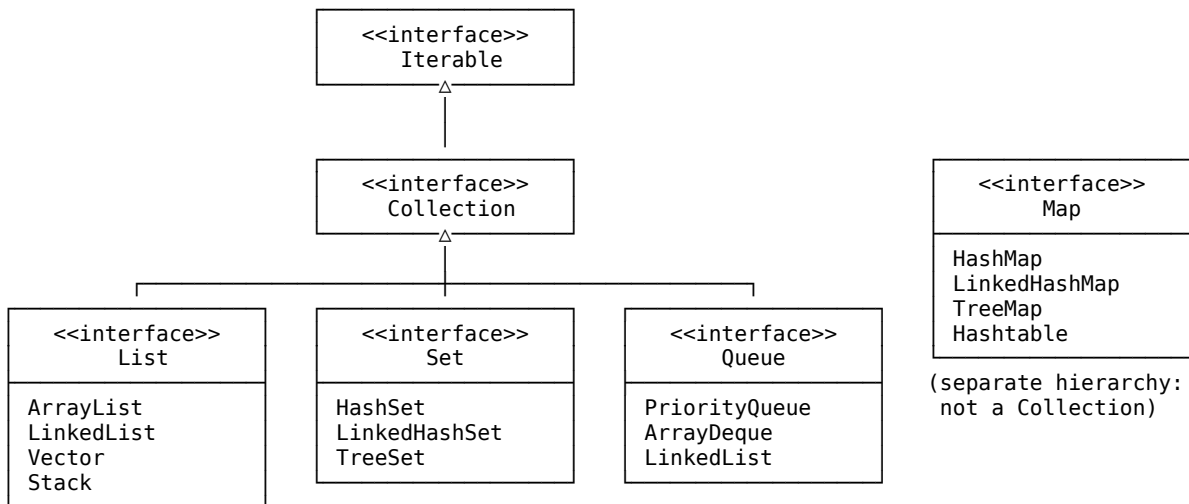


Figure 2: List branch - ArrayList, LinkedList, Vector and Stack

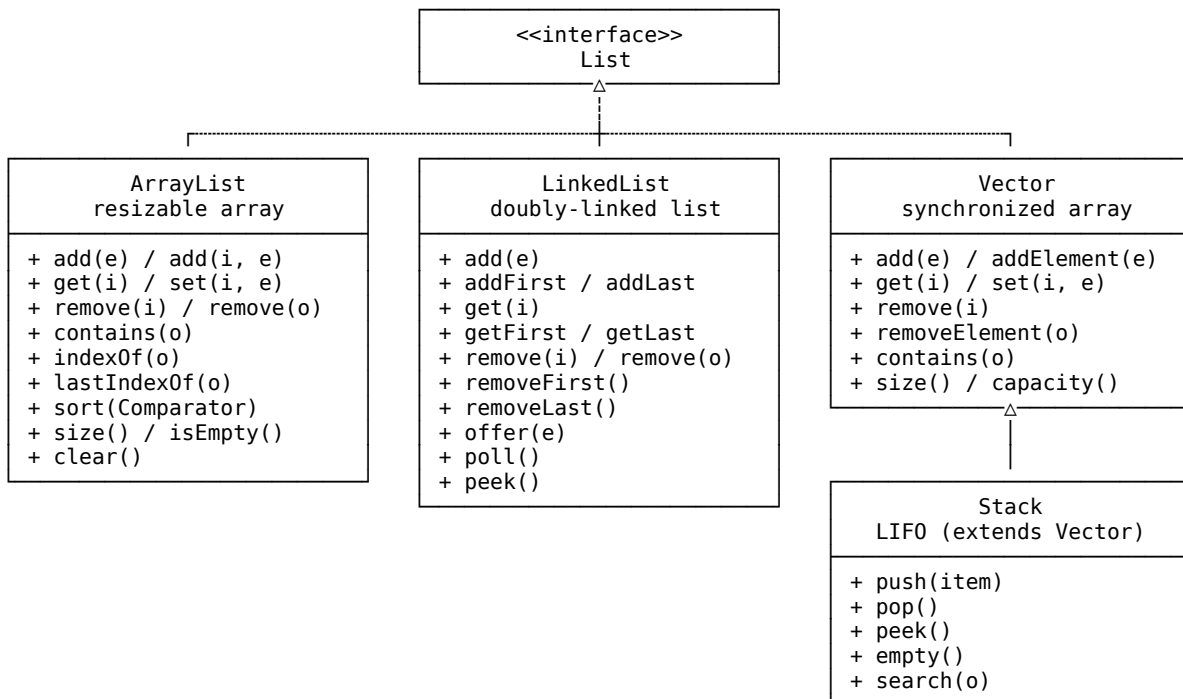


Figure 3: Set branch - HashSet, LinkedHashSet and TreeSet

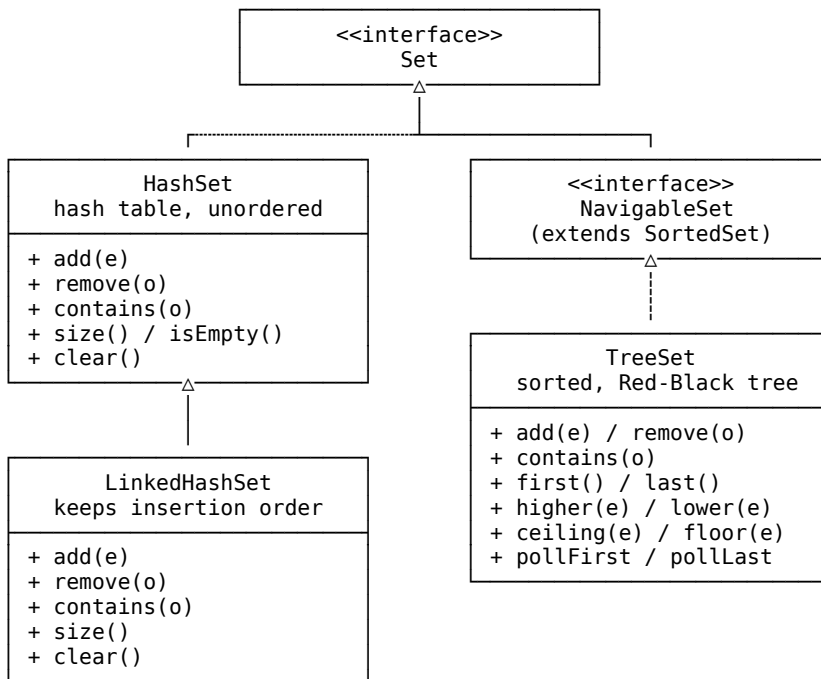


Figure 4: Queue / Deque branch - PriorityQueue, ArrayDeque and LinkedList

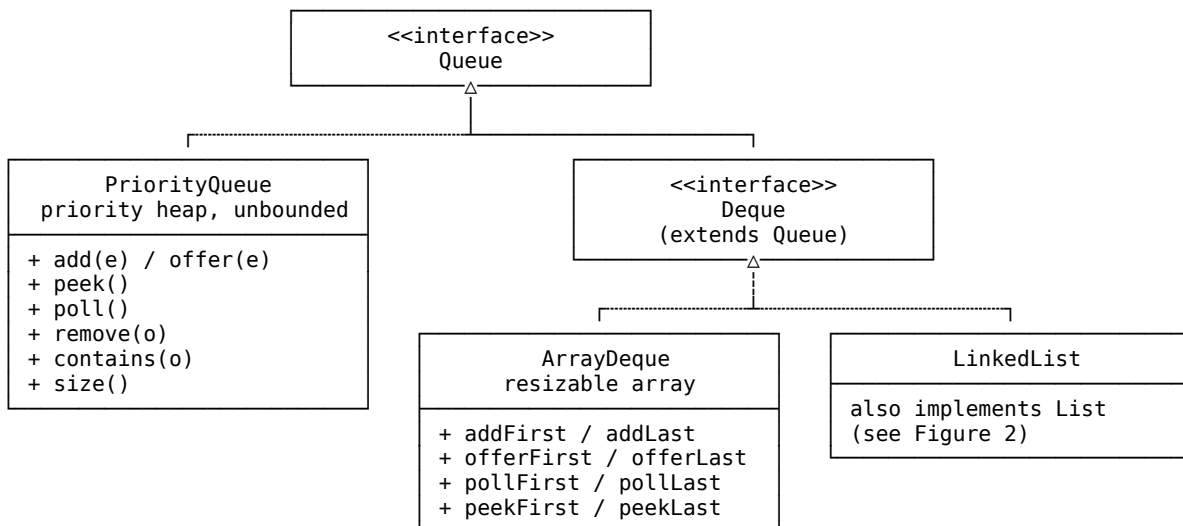
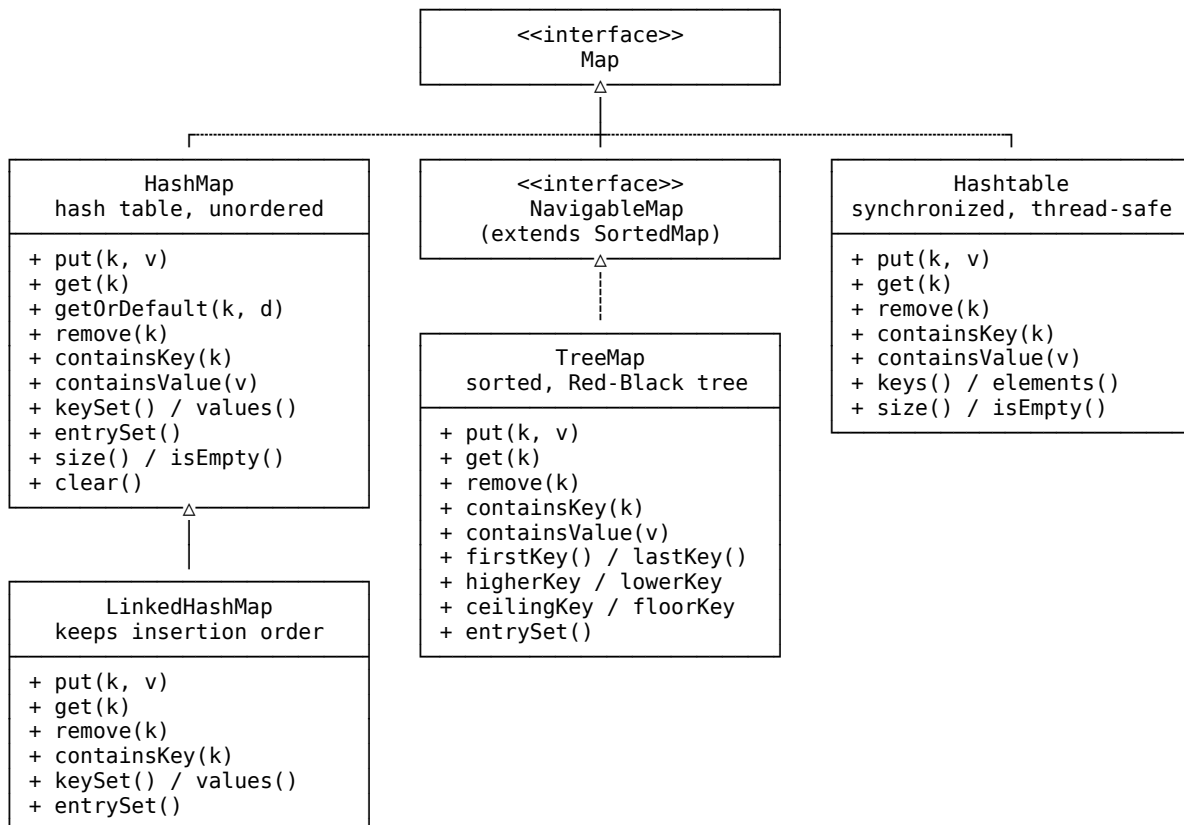


Figure 5: Map branch - HashMap, LinkedHashMap, TreeMap and Hashtable



Legend

- △ solid line with hollow triangle : extends (inheritance)
- - -△ dashed line with hollow triangle : implements (realization)
- <<interface>> : Java interface
- + method() : public method

Relationships

- "List", "Set" and "Queue" extend "Collection", which extends "Iterable". "Map" is a separate hierarchy and does not extend "Collection".
- "ArrayList", "LinkedList" and "Vector" implement "List". "Stack" extends "Vector".
- "LinkedList" implements both "List" and "Deque".
- "HashSet" implements "Set". "LinkedHashSet" extends "HashSet". "TreeSet" implements "NavigableSet" (which extends "SortedSet" and "Set").
- "PriorityQueue" implements "Queue". "ArrayDeque" implements "Deque" (which extends "Queue").
- "HashMap" and "Hashtable" implement "Map". "LinkedHashMap" extends "HashMap". "TreeMap" implements "NavigableMap" (which extends "SortedMap" and "Map").

6. Source Code

```

import java.util.*;

public class CollectionDemo {

    static void header(String title) {
        System.out.println("\n==== " + title + " =====");
    }

    // ----- ArrayList -----
    static void arrayListDemo() {
        header("ArrayList");
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        list.add(1, "Mango");
    }
}
  
```

```

        System.out.println("After add()           : " + list);
        System.out.println("get(2)               : " + list.get(2));
        list.set(0, "Orange");
        System.out.println("After set(0, Orange) : " + list);
        list.add("Banana");
        System.out.println("indexOf(Banana)       : " + list.indexOf("Banana"));
        System.out.println("lastIndexOf(Banana)  : " + list.lastIndexOf("Banana"));
        list.remove(1);
        list.remove("Cherry");
        System.out.println("After remove()       : " + list);
        System.out.println("contains(Banana)     : " + list.contains("Banana"));
        list.sort(Comparator.naturalOrder());
        System.out.println("After sort()         : " + list);
        System.out.println("size()               : " + list.size());
        list.clear();
        System.out.println("isEmpty() after clear(): " + list.isEmpty());
    }

    // ----- LinkedList -----
    static void linkedListDemo() {
        header("LinkedList");
        LinkedList<String> ll = new LinkedList<>();
        ll.add("B");
        ll.addFirst("A");
        ll.addLast("D");
        ll.add(2, "C");
        System.out.println("After add methods    : " + ll);
        System.out.println("get(1)              : " + ll.get(1));
        System.out.println("getFirst() / getLast() : " + ll.getFirst() + " / " + ll.getLast());
        ll.offer("E");
        System.out.println("After offer(E)       : " + ll);
        System.out.println("removeFirst()        : " + ll.removeFirst());
        System.out.println("removeLast()         : " + ll.removeLast());
        ll.remove("C");
        ll.remove(0);
        System.out.println("After remove()       : " + ll);
        System.out.println("peek()               : " + ll.peek());
        System.out.println("poll()               : " + ll.poll());
        System.out.println("poll() on empty list : " + ll.poll());
    }

    // ----- Vector -----
    static void vectorDemo() {
        header("Vector");
        Vector<Integer> v = new Vector<>();
        System.out.println("Initial capacity()   : " + v.capacity());
        v.add(10);
        v.addElement(20);
        v.add(30);
        System.out.println("After add/addElement : " + v);
        System.out.println("get(1)               : " + v.get(1));
        v.set(1, 25);
        System.out.println("After set(1, 25)     : " + v);
        v.remove(0);
        v.removeElement(30);
        System.out.println("After remove methods : " + v);
        System.out.println("contains(25)         : " + v.contains(25));
        System.out.println("size()               : " + v.size());
    }

    // ----- Stack -----
    static void stackDemo() {
        header("Stack");
        Stack<String> st = new Stack<>();
        st.push("Java");
        st.push("Python");
        st.push("C++");
        System.out.println("Stack                : " + st);
        System.out.println("peek()               : " + st.peek());
        System.out.println("search(Java)         : " + st.search("Java"));
        System.out.println("pop()                : " + st.pop());
        System.out.println("After pop()          : " + st);
        System.out.println("empty()              : " + st.empty());
    }

    // ----- HashSet -----
    static void hashSetDemo() {
        header("HashSet");
    }

```

```

HashSet<String> hs = new HashSet<>();
hs.add("Red");
hs.add("Green");
hs.add("Blue");
System.out.println("add(Red) again      : " + hs.add("Red"));
System.out.println("Set (no duplicates) : " + hs);
System.out.println("contains(Green)     : " + hs.contains("Green"));
hs.remove("Blue");
System.out.println("After remove(Blue)   : " + hs);
System.out.println("size() / isEmpty()   : " + hs.size() + " / " + hs.isEmpty());
hs.clear();
System.out.println("After clear()        : " + hs);
}

// ----- HashSet -----
static void linkedHashSetDemo() {
    header("LinkedHashSet");
    LinkedHashSet<String> lhs = new LinkedHashSet<>();
    lhs.add("Zebra");
    lhs.add("Apple");
    lhs.add("Mango");
    lhs.add("Apple");
    System.out.println("Insertion order kept : " + lhs);
    System.out.println("contains(Mango)      : " + lhs.contains("Mango"));
    lhs.remove("Zebra");
    System.out.println("After remove(Zebra)   : " + lhs);
    System.out.println("size()              : " + lhs.size());
    lhs.clear();
    System.out.println("After clear()        : " + lhs);
}

// ----- TreeSet -----
static void treeSetDemo() {
    header("TreeSet");
    TreeSet<Integer> ts = new TreeSet<>();
    ts.add(40);
    ts.add(10);
    ts.add(30);
    ts.add(20);
    ts.add(50);
    System.out.println("Sorted set          : " + ts);
    System.out.println("first() / last()     : " + ts.first() + " / " + ts.last());
    System.out.println("higher(30) / lower(30) : " + ts.higher(30) + " / " + ts.lower(30));
    System.out.println("ceiling(25) / floor(25) : " + ts.ceiling(25) + " / " + ts.floor(25));
    System.out.println("contains(20)         : " + ts.contains(20));
    ts.remove(20);
    System.out.println("After remove(20)     : " + ts);
    System.out.println("pollFirst()          : " + ts.pollFirst());
    System.out.println("pollLast()           : " + ts.pollLast());
    System.out.println("Remaining             : " + ts);
}

// ----- PriorityQueue -----
static void priorityQueueDemo() {
    header("PriorityQueue");
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    pq.add(30);
    pq.offer(10);
    pq.add(20);
    pq.offer(5);
    System.out.println("peek() (smallest)    : " + pq.peek());
    System.out.println("contains(20)         : " + pq.contains(20));
    pq.remove(20);
    System.out.println("size() after remove  : " + pq.size());
    System.out.print("poll() order        : ");
    while (!pq.isEmpty()) {
        System.out.print(pq.poll() + " ");
    }
    System.out.println();
    System.out.println("poll() on empty queue : " + pq.poll());
}

// ----- ArrayDeque -----
static void arrayDequeDemo() {
    header("ArrayDeque");
    ArrayDeque<String> dq = new ArrayDeque<>();
    dq.addFirst("B");
    dq.addLast("C");
}

```

```

        dq.offerFirst("A");
        dq.offerLast("D");
        System.out.println("Deque                : " + dq);
        System.out.println("peekFirst() + " / " + dq.peekLast());
        System.out.println("pollFirst()      : " + dq.pollFirst());
        System.out.println("pollLast()       : " + dq.pollLast());
        System.out.println("Remaining        : " + dq);
    }

// ----- HashMap -----
static void hashMapDemo() {
    header("HashMap");
    HashMap<Integer, String> hm = new HashMap<>();
    hm.put(1, "Rahul");
    hm.put(2, "Priya");
    hm.put(3, "Amit");
    hm.put(2, "Sneha");
    System.out.println("Map (key 2 replaced) : " + hm);
    System.out.println("get(3)                : " + hm.get(3));
    System.out.println("get(9)                : " + hm.get(9));
    System.out.println("getOrDefault(9)      : " + hm.getOrDefault(9, "Not Found"));
    System.out.println("containsKey(1)        : " + hm.containsKey(1));
    System.out.println("containsValue(Amit)   : " + hm.containsValue("Amit"));
    System.out.println("keySet()              : " + hm.keySet());
    System.out.println("values()              : " + hm.values());
    System.out.println("entrySet()            : " + hm.entrySet());
    hm.remove(1);
    System.out.println("After remove(1)       : " + hm);
    System.out.println("size() / isEmpty()    : " + hm.size() + " / " + hm.isEmpty());
    hm.clear();
    System.out.println("After clear()         : " + hm);
}

// ----- LinkedHashMap -----
static void linkedHashMapDemo() {
    header("LinkedHashMap");
    LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>();
    lhm.put("Maths", 90);
    lhm.put("Physics", 85);
    lhm.put("Chemistry", 88);
    System.out.println("Insertion order kept : " + lhm);
    System.out.println("get(Physics)          : " + lhm.get("Physics"));
    System.out.println("containsKey(Maths)    : " + lhm.containsKey("Maths"));
    lhm.remove("Physics");
    System.out.println("keySet()              : " + lhm.keySet());
    System.out.println("values()              : " + lhm.values());
    System.out.println("entrySet()            : " + lhm.entrySet());
}

// ----- TreeMap -----
static void treeMapDemo() {
    header("TreeMap");
    TreeMap<Integer, String> tm = new TreeMap<>();
    tm.put(30, "C");
    tm.put(10, "A");
    tm.put(50, "E");
    tm.put(20, "B");
    tm.put(40, "D");
    System.out.println("Sorted by key         : " + tm);
    System.out.println("get(20)               : " + tm.get(20));
    System.out.println("firstKey() / lastKey() : " + tm.firstKey() + " / " + tm.lastKey());
    System.out.println("higherKey(30)/lowerKey : " + tm.higherKey(30) + " / " + tm.lowerKey(30));
    System.out.println("ceilingKey(25)/floorKey : " + tm.ceilingKey(25) + " / " + tm.floorKey(25));
    System.out.println("containsKey(40)       : " + tm.containsKey(40));
    System.out.println("containsValue(Z)      : " + tm.containsValue("Z"));
    tm.remove(10);
    System.out.println("After remove(10)      : " + tm);
    System.out.println("entrySet()            : " + tm.entrySet());
}

// ----- Hashtable -----
static void hashtableDemo() {
    header("Hashtable");
    Hashtable<String, Integer> ht = new Hashtable<>();
    ht.put("One", 1);
    ht.put("Two", 2);
    ht.put("Three", 3);
    System.out.println("get(Two)              : " + ht.get("Two"));
}

```

```

        System.out.println("containsKey(One)      : " + ht.containsKey("One"));
        System.out.println("containsValue(3)      : " + ht.containsValue(3));
        System.out.print("keys()                  : ");
        Enumeration<String> keys = ht.keys();
        while (keys.hasMoreElements()) {
            System.out.print(keys.nextElement() + " ");
        }
        System.out.println();
        System.out.print("elements()              : ");
        Enumeration<Integer> vals = ht.elements();
        while (vals.hasMoreElements()) {
            System.out.print(vals.nextElement() + " ");
        }
        System.out.println();
        ht.remove("One");
        System.out.println("After remove(One)      : " + ht);
        System.out.println("size() / isEmpty()    : " + ht.size() + " / " + ht.isEmpty());
    }

    public static void main(String[] args) {
        arrayListDemo();
        linkedListDemo();
        vectorDemo();
        stackDemo();
        hashSetDemo();
        linkedHashSetDemo();
        treeSetDemo();
        priorityQueueDemo();
        arrayDequeDemo();
        hashMapDemo();
        linkedHashMapDemo();
        treeMapDemo();
        hashtableDemo();
    }
}

```

7. Compilation & Execution Commands

Save the source code exactly as:

CollectionDemo.java

Compile the program:

```
javac CollectionDemo.java
```

Run the compiled program:

```
java CollectionDemo
```

8. Output Screen

Program output captured from the Eclipse IDE console (JDK run of CollectionDemo):

```
<terminated> CollectionDemo [Java Application] C:\Users\togid\Downloads\ eclipse-jee-20
===== ArrayList =====
After add() : [Apple, Mango, Banana, Cherry]
get(2) : Banana
After set(0, Orange) : [Orange, Mango, Banana, Cherry]
indexOf(Banana) : 2
lastIndexOf(Banana) : 4
After remove() : [Orange, Banana, Banana]
contains(Banana) : true
After sort() : [Banana, Banana, Orange]
size() : 3
isEmpty() after clear(): true

===== LinkedList =====
After add methods : [A, B, C, D]
get(1) : B
getFirst() / getLast() : A / D
After offer(E) : [A, B, C, D, E]
removeFirst() : A
removeLast() : E
After remove() : [D]
peek() : D
poll() : D
poll() on empty list : null

===== Vector =====
Initial capacity() : 10
After add/addElement : [10, 20, 30]
get(1) : 20
After set(1, 25) : [10, 25, 30]
After remove methods : [25]
contains(25) : true
size() : 1

===== Stack =====
Stack : [Java, Python, C++]
peek() : C++
search(Java) : 3
pop() : C++
After pop() : [Java, Python]
empty() : false

REDDMI 14C 5G 03/10/2026 21:18
```

Screenshot 1: ArrayList to Stack

```
<terminated> CollectionDemo [Java Application] C:\Users\togid\Downloads\ eclipse-jee-20
===== HashSet =====
add(Red) again : false
Set (no duplicates) : [Red, Blue, Green]
contains(Green) : true
After remove(Blue) : [Red, Green]
size() / isEmpty() : 2 / false
After clear() : []

===== LinkedHashSet =====
Insertion order kept : [Zebra, Apple, Mango]
contains(Mango) : true
After remove(Zebra) : [Apple, Mango]
size() : 2
After clear() : []

===== TreeSet =====
Sorted set : [10, 20, 30, 40, 50]
first() / last() : 10 / 50
higher(30) / lower(30) : 40 / 20
ceiling(25) / floor(25) : 30 / 20
contains(20) : true
After remove(20) : [10, 30, 40, 50]
pollFirst() : 10
pollLast() : 50
Remaining : [30, 40]

===== PriorityQueue =====
peek() (smallest) : 5
contains(20) : true
size() after remove : 3
poll() order : 5 10 30
poll() on empty queue : null

===== ArrayDeque =====
Deque : [A, B, C, D]
peekFirst / peekLast : A / D
pollFirst() : A
pollLast() : D
Remaining : [B, C]

===== HashMap =====

REDDMI 14C 5G 03/10/2026 21:18
```

Screenshot 2: HashSet to ArrayDeque

```
Problems Javadoc Declaration Console X History
<terminated> CollectionDemo [Java Application] C:\Users\togid\Downloads\ eclipse-jee-2025-0
Map (key 2 replaced) : {1=Rahul, 2=Sneha, 3=Amit}
get(3) : Amit
get(9) : null
getOrDefault(9) : Not Found
containsKey(1) : true
containsValue(Amit) : true
keySet() : [1, 2, 3]
values() : [Rahul, Sneha, Amit]
entrySet() : [1=Rahul, 2=Sneha, 3=Amit]
After remove(1) : {2=Sneha, 3=Amit}
size() / isEmpty() : 2 / false
After clear() : {}

===== LinkedHashMap =====
Insertion order kept : {Maths=90, Physics=85, Chemistry=88}
get(Physics) : 85
containsKey(Maths) : true
keySet() : [Maths, Chemistry]
values() : [90, 88]
entrySet() : [Maths=90, Chemistry=88]

===== TreeMap =====
Sorted by key : {10=A, 20=B, 30=C, 40=D, 50=E}
get(20) : B
firstKey() / lastKey() : 10 / 50
higherKey(30) / lowerKey : 40 / 20
ceilingKey(25) / floorKey : 30 / 20
containsKey(40) : true
containsValue(2) : false
After remove(10) : {20=B, 30=C, 40=D, 50=E}
entrySet() : [20=B, 30=C, 40=D, 50=E]

===== Hashtable =====
get(Two) : 2
containsKey(One) : true
containsValue(3) : true
keys() : One Three Two
elements() : 1 3 2
After remove(One) : {Three=3, Two=2}
size() / isEmpty() : 2 / false

REDDMI 14C 5G 03/10/2026 21:19
```

Screenshot 3: HashMap to Hashtable

Console output (text)

```
===== ArrayList =====
After add() : [Apple, Mango, Banana, Cherry]
get(2) : Banana
After set(0, Orange) : [Orange, Mango, Banana, Cherry]
indexOf(Banana) : 2
lastIndexOf(Banana) : 4
After remove() : [Orange, Banana, Banana]
contains(Banana) : true
After sort() : [Banana, Banana, Orange]
size() : 3
isEmpty() after clear(): true

===== LinkedList =====
After add methods : [A, B, C, D]
get(1) : B
getFirst() / getLast() : A / D
After offer(E) : [A, B, C, D, E]
removeFirst() : A
removeLast() : E
After remove() : [D]
peek() : D
poll() : D
poll() on empty list : null

===== Vector =====
Initial capacity() : 10
After add/addElement : [10, 20, 30]
get(1) : 20
After set(1, 25) : [10, 25, 30]
After remove methods : [25]
contains(25) : true
size() : 1

===== Stack =====
Stack : [Java, Python, C++]
peek() : C++
search(Java) : 3
pop() : C++
After pop() : [Java, Python]
empty() : false

===== HashSet =====
add(Red) again : false
Set (no duplicates) : [Red, Blue, Green]
contains(Green) : true
After remove(Blue) : [Red, Green]
size() / isEmpty() : 2 / false
After clear() : []

===== LinkedHashSet =====
Insertion order kept : [Zebra, Apple, Mango]
contains(Mango) : true
After remove(Zebra) : [Apple, Mango]
size() : 2
After clear() : []

===== TreeSet =====
Sorted set : [10, 20, 30, 40, 50]
first() / last() : 10 / 50
higher(30) / lower(30) : 40 / 20
ceiling(25) / floor(25) : 30 / 20
contains(20) : true
After remove(20) : [10, 30, 40, 50]
pollFirst() : 10
pollLast() : 50
Remaining : [30, 40]

===== PriorityQueue =====
peek() (smallest) : 5
contains(20) : true
size() after remove : 3
poll() order : 5 10 30
poll() on empty queue : null

===== ArrayDeque =====
Deque : [A, B, C, D]
```

```

peekFirst / peekLast : A / D
pollFirst()          : A
pollLast()           : D
Remaining             : [B, C]

===== HashMap =====
Map (key 2 replaced) : {1=Rahul, 2=Sneha, 3=Amit}
get(3)               : Amit
get(9)               : null
getOrDefault(9)      : Not Found
containsKey(1)        : true
containsValue(Amit)   : true
keySet()              : [1, 2, 3]
values()              : [Rahul, Sneha, Amit]
entrySet()            : [1=Rahul, 2=Sneha, 3=Amit]
After remove(1)       : {2=Sneha, 3=Amit}
size() / isEmpty()    : 2 / false
After clear()         : {}

===== LinkedHashMap =====
Insertion order kept : {Maths=90, Physics=85, Chemistry=88}
get(Physics)          : 85
containsKey(Maths)    : true
keySet()              : [Maths, Chemistry]
values()              : [90, 88]
entrySet()            : [Maths=90, Chemistry=88]

===== TreeMap =====
Sorted by key         : {10=A, 20=B, 30=C, 40=D, 50=E}
get(20)               : B
firstKey() / lastKey() : 10 / 50
higherKey(30)/lowerKey : 40 / 20
ceilingKey(25)/floorKey : 30 / 20
containsKey(40)       : true
containsValue(Z)       : false
After remove(10)      : {20=B, 30=C, 40=D, 50=E}
entrySet()            : [20=B, 30=C, 40=D, 50=E]

===== Hashtable =====
get(Two)              : 2
containsKey(One)       : true
containsValue(3)       : true
keys()                 : One Three Two
elements()             : 1 3 2
After remove(One)      : {Three=3, Two=2}
size() / isEmpty()     : 2 / false

```

9. Observation Table

Class	Ordering	Duplicates	Null allowed	Thread-safe
ArrayList	Insertion order	Yes	Yes	No
LinkedList	Insertion order	Yes	Yes	No
Vector	Insertion order	Yes	Yes	Yes
Stack	LIFO	Yes	Yes	Yes
HashSet	Unordered	No	One null	No
LinkedHashSet	Insertion order	No	One null	No
TreeSet	Sorted	No	No	No
PriorityQueue	Priority (heap)	Yes	No	No
ArrayDeque	Insertion order	Yes	No	No
HashMap	Unordered keys	Unique keys	One null key	No
LinkedHashMap	Insertion order	Unique keys	One null key	No
TreeMap	Sorted keys	Unique keys	No null key	No
Hashtable	Unordered keys	Unique keys	No	Yes

10. Result

The important methods of the Java Collection Framework classes were implemented and executed successfully. The program demonstrated list operations, set behaviour (duplicate rejection, insertion order and sorted order), queue and deque operations, and map operations such as `put()`, `get()`, `getOrDefault()`, `keySet()`, `values()` and `entrySet()`.